2MA100 - TP final

Ce TP final est prévu pour être effectué en **2h** : il n'est pas nécaissaire d'avoir fait les trois exercices pour avoir une note correcte. Les exercices doivent être rendus au choix au format **Notebook** (`.ipynb`) ou **Script** (`.py`) en un ou plusieurs fichiers au plus tard le **21 mai 2020 à 18h** sur Moodle. Il ne faut pas indiquer votre prénom, nom ou numéro étudiant dans ce qui est rendu sur Moodle, y compris dans le nom des fichiers afin de garantir votre anonymat lors de la correction. Il n'est pas nécaissaire d'indiquer le numéro d'anonymat de votre convocation d'examen, Moodle se charge d'anonymiser les copies automatiquement.

Il est possible de rendre vos exercices jusqu'à 22h, mais alors le rendu se fait **sous votre propre responsabilité** en cas de défaillance technique (Moodle, ordinateur ou autre) ne vous permettant pas de rendre vos exercices. En cas de problème technique de 9h à 18h vous pouvez envoyer un email à julien.guillod@sorbonne-universite.fr en mentionnant `[2MA100 examen]` dans le sujet de votre email.

L'accès aux sites Internet suivants est autorisé :

- Site du cours : `https://python.guillod.org/`
- Carte de référence Python : `https://python.guillod.org/carteref.pdf`
- Documentation de Python : `https://docs.python.org/`
- Documentation de Numpy : `https://docs.scipy.org/`
- Documentation de Matplotlib : `https://matplotlib.org/`
- Documentation de Sympy : `https://docs.sympy.org/`

L'accès à d'autres sites Internet ou ressources (livres, cours, ou autre) ainsi que la communication avec qui que ce soit ne sont pas autorisés. Les tentatives de fraudes seront dénoncées à la section disciplinaire avec les conséquences indiquées dans les modalités de contrôles des connaissances (article 8).

Les modules Numpy et Matplotlib peuvent être utiles dans certains exercices :

```
import numpy as np
import matplotlib.pyplot as plt
```

Exercice 1 : Formule de Leibniz

On rappelle la formule de Leibniz :

$$\frac{\pi}{4} = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1},$$

et le but est de l'utiliser pour obtenir une approximation de π. Pour un entier $n \geq 0$, on note

$$S_n = \sum_{k=0}^{n} \frac{(-1)^k}{2k+1}.$$

a) Écrire une fonction qui, à n donné, calcule S_n.

b) Écrire une fonction dépendant de ε qui renvoie le plus petit entier n tel que $|\pi - 4S_n| < \varepsilon$.

c) Tracer l'évolution de $|\pi - 4S_n|$ en fonction de n pour $3 \leq n \leq 100$.

d) Vos implémentations des deux questions précédentes sont-elles optimales quand au nombre d'opérations effectuées ?

Exercice 2 : Equation différentielle

Considérons l'équation différentielle linéaire suivante

$$\begin{cases} RC\,\dot{u}(t) + u(t) = 0, \\ u(0) = u_0 \end{cases}$$

qui modélise la tension aux bornes d'un condensateur dans un circuit RC (c'est-à-dire composé d'une résistance et d'un condensateur montés en série). Le symbol RC est considéré ici comme une constante, et dans les applications numériques on prendra $RC = 0.1$.

a) Ecrire une fonction `sol_exacte(t,u0,RC)` retournant la solution exacte donnée par

$$u(t) = u_0 \exp\left(-\frac{t}{RC}\right).$$

b) Donner la fonction $f : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$ telle que l'équation différentielle ci-dessus s'écrit sous la forme canonique $\dot{u}(t) = f(t, u(t))$ et implémenter cette fonction f en Python.

c) Écrire une fonction permettant de déterminer une solution approchée de l'équation différentielle avec la méthode d'Euler explicite :

$$u_{n+1} = u_n + \Delta t f(t_n, u_n).$$

Pour la donnée initiale $u_0 = 1$ et le temps final $T = 1$, comparer graphiquement la courbe obtenue avec la solution exacte, celle obtenue avec la solution approchée pour un pas de temps $\Delta t = 0.1$ et celle obtenue pour $\Delta t = 0.01$.

d) Écrire une fonction permettant de déterminer une solution approchée avec une méthode de Runge Kutta d'ordre deux (RK2) donnée par

$$u_{n+1} = u_n + \frac{\Delta t}{2}\big[f(t_n, u_n) + f(t_{n+1}, u_*)\big]$$

où

$$u_* = u_n + \Delta t f(t_n, u_n).$$

e) Étude qualitative : comparer graphiquement la courbe obtenue avec la solution exacte, celle obtenue avec la solution approchée par RK2 pour $\Delta t = 0.1$ et celle obtenue pour $\Delta t = 0.01$, toujours avec $T = 1$ et $u_0 = 1$, puis commenter.

f) Étude quantitative : calculer l'erreur entre la solution exacte et une solution approchée au temps final $T = 1$:

$$E_N = |u(T) - u_N|$$

pour un pas de temps $\Delta t = T/N$ avec N entier pour les mêmes données initiales que précédemment. Tracer en échelles logarithmiques la courbe d'erreur en fonction de Δt pour les valeurs $N = 10, 100, 1000, 10000, 100000$ et pour les deux méthodes numériques (Euler explciite et RK2). Comparer ces deux courbes avec les courbes $\sqrt{\Delta t}$, Δt et Δt^2 et commenter.

Exercice 3 : Interpolation polynomiale

Le but de cet exercice est d'approximer une fonction par un polynôme par la méthode dite des différences divisées. Pour cela, on considère $n+1$ points distincts x_0, x_1, ..., x_n et on dispose de $n+1$ valeurs $(y_0, y_1, \ldots, y_n)$ de f telles que $y_i = f(x_i)$ pour $i \in \{0, 1, \ldots, n\}$. Le but est de déterminer le polynôme P_n tel que $P_n(x_i) = y_i$, *i.e.* le polynôme d'interpolation de Lagrange. Pour ce faire, on introduit les différences divisées définies par :

$$[\![y_i]\!] = y_i\,, \quad \forall i \in \{0, 1, \ldots, n\}\,,$$

puis à l'ordre 1 par :

$$[\![y_i, y_{i+1}]\!] = \frac{[\![y_{i+1}]\!] - [\![y_i]\!]}{x_{i+1} - x_i}\,, \quad \forall i \in \{0, \ldots, n-1\}\,,$$

puis à l'ordre 2 par :

$$[\![y_i, y_{i+1}, y_{i+2}]\!] = \frac{[\![y_{i+1}, y_{i+2}]\!] - [\![y_i, y_{i+1}]\!]}{x_{i+2} - x_i}\,, \quad \forall i \in \{0, \ldots, n-2\}\,,$$

et ainsi de suite jusqu'à :

$$[\![y_0, y_1, \ldots, y_n]\!] = \frac{[\![y_1, y_2, \ldots, y_n]\!] - [\![y_0, y_1, \ldots, y_{n-1}]\!]}{x_n - x_0}\,.$$

Récursivement les différences divisées sont données par :

$$[\![y_i, \ldots, y_j]\!] = \frac{[\![y_{i+1}, \ldots, y_j]\!] - [\![y_i, \ldots, y_{j-1}]\!]}{x_j - x_i}\,,$$

pour $0 \leq i < j \leq n$. Le polynôme d'interpolation de Lagrange est alors donné par :

$$P_n(x) = [\![y_0]\!] + \sum_{i=1}^{n} [\![y_0, \ldots, y_i]\!](x - x_0)(x - x_1)\cdots(x - x_{i-1})$$

Pour programmer cet algorithme, on remplit un tableau triangulaire à double entrée : à chaque étape on remplit une colonne supplémentaire.

étape	$t = 0$	$t = 1$	$t = 2$	$\cdots$	$t = n$
tableau[0]	$[\![y_0]\!]$				
tableau[1]	$[\![y_1]\!]$	$[\![y_0, y_1]\!]$			
tableau[2]	$[\![y_2]\!]$	$[\![y_1, y_2]\!]$	$[\![y_0, y_1, y_2]\!]$		
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	
tableau[n]	$[\![y_n]\!]$	$[\![y_{n-1}, y_n]\!]$	$[\![y_{n-2}, y_{n-1}, y_n]\!]$		$[\![y_0, \ldots, y_n]\!]$

a) Écrire une fonction `differences(x_interp,y_interp)` qui prend en argument les listes ou vecteurs $(x_0, \ldots, x_n)$ et $(y_0, \ldots, y_n)$ et qui retournent la liste ou le vecteur des différences divisées $([\![y_0]\!], \ldots, [\![y_0, \cdots, y_n]\!])$.

b) Écrire une fonction `interpole(a,x_interp,liste)` qui calcule la valeur de P_n en un point $a \in \mathbb{R}$ où `x_interp` représente $(x_0, \ldots, x_n)$ et `liste` est la sortie de la fontion `differences`.

c) On considère la fonction $f(x) = 1/(1 + 10x^2)$. Construire l'approximation polynomiale P_n de f par interpolation en cinq points régulièrement repartis dans l'intervalle $[0, 1]$. Tracer sur la même figure, les graphes de la fonction f et du polynôme P_n en prenant 50 points pour la discrétisation des fonctions.